

Support de cours

IMPLEMENTATION DES BASES DE DONNEES LIBRES (MYSQL, POSTGRESQL, MARIADB)

Préface

La maîtrise des bases de données constitue aujourd'hui une compétence essentielle pour tout professionnel de l'informatique, de la gestion des systèmes d'information et du développement d'applications. Dans un contexte où les volumes de données croissent de manière exponentielle et où la disponibilité des informations est cruciale, savoir concevoir, implémenter et administrer des bases de données fiables et performantes est devenu indispensable.

Le présent cours, intitulé ***Implémentation et Administration des Bases de Données Libres***, s'inscrit dans cette dynamique. Il propose une approche pratique et progressive des systèmes de gestion de bases de données relationnelles libres les plus utilisés : **MySQL, MariaDB et PostgreSQL**. Ces outils offrent une combinaison unique de robustesse, de performance et de flexibilité, tout en étant accessibles et largement déployés dans l'industrie et le secteur académique.

L'objectif de ce cours est double :

- **Théorique** : fournir une compréhension solide des concepts fondamentaux des bases de données relationnelles, des modèles de données, des transactions et de la sécurité.
- **Pratique** : permettre aux étudiants de manipuler directement les SGBD, d'implémenter des bases complètes, de rédiger des requêtes simples et complexes, de créer des procédures automatisées et de gérer l'administration et la sécurité des systèmes.

Chaque module a été conçu pour être à la fois **pédagogique et opérationnel**, avec des exemples concrets, des travaux pratiques et des études de cas reflétant les situations professionnelles réelles. Le cours couvre toutes les étapes, de la modélisation conceptuelle à l'optimisation des requêtes et à l'administration complète des bases de données, incluant sauvegarde, restauration et monitoring.

En suivant ce cours, l'étudiant sera en mesure de concevoir et de gérer efficacement des bases de données, de garantir leur intégrité et leur performance, et de faire des choix éclairés entre différents systèmes selon les besoins d'un projet. Ce parcours constitue ainsi une préparation solide à la pratique professionnelle et à l'exploitation des bases de données libres dans un environnement réel.

Nous espérons que ce cours stimulera votre curiosité, renforcera vos compétences et vous donnera la confiance nécessaire pour relever les défis liés à la gestion des données dans le monde numérique actuel.

AM. HOUNGBO

Module 1- Introduction générale aux Systèmes de Gestion de Bases de Données Libres (SGBD libres) – Historique, principes de l'open source et présentation de MySQL, PostgreSQL et MariaDB.

🧩 1. Introduction générale aux bases de données

1.1. Définition

Une **base de données** est un ensemble organisé et structuré d'informations, stockées de manière à pouvoir être facilement consultées, modifiées et gérées. Le **Système de Gestion de Base de Données (SGBD)** est le logiciel qui permet de créer, manipuler et administrer ces données.

Exemples de SGBD :

- **Propriétaires** : Oracle, SQL Server, IBM DB2, Microsoft Access.
- **Libres / Open Source** : MySQL, PostgreSQL, MariaDB, SQLite.

1.2. Rôle d'un SGBD

Un SGBD permet :

- de **créer** et **gérer** des bases de données,
- de **contrôler l'accès** aux données (sécurité, utilisateurs, droits),
- d'**assurer l'intégrité et la cohérence** des informations,
- de **partager les données** entre plusieurs utilisateurs ou applications,
- d'**optimiser les performances** et de garantir la **fiabilité** des transactions.

🕰️ 2. Historique des SGBD

Période	Évolution majeure
Années 1960-1970	Apparition des premiers SGBD hiérarchiques et réseaux (IMS, IDMS).
Années 1980	Émergence du **modèle relationnel** (Edgar F. Codd) et du SQL.
Années 1990	Généralisation des SGBD relationnels propriétaires (Oracle, SQL Server).
Années 2000	Explosion des **SGBD open source** avec Internet (MySQL, PostgreSQL).
Années 2010-2020	Apparition des SGBD **NoSQL** et hybridation avec les modèles relationnels.
Aujourd'hui	Consolidation des solutions libres et open source pour le cloud, le big data et le web.

🌍 3. Principes de l'open source appliqués aux SGBD

3.1. Définition du logiciel libre

Un **logiciel libre** (ou **open source**) est un programme dont le **code source est accessible** et que tout utilisateur peut :

- **Utiliser** librement,
- **Étudier et modifier**,
- **Redistribuer** avec ou sans modifications.

Ces libertés sont garanties par des licences comme :

- **GNU GPL (General Public License)**
- **MIT License**
- **BSD License**

3.2. Avantages des SGBD libres

- **Gratuité d'utilisation** (pas de coût de licence).
- **Transparence** du code et **communauté active**.
- **Indépendance vis-à-vis d'un fournisseur**.
- **Personnalisation** et **interopérabilité**.

- **Mises à jour régulières** et support communautaire.

3.3. Inconvénients potentiels

- Support parfois communautaire et non garanti.
- Nécessité de compétences techniques pour l'installation et la maintenance.
- Hétérogénéité des outils graphiques selon les distributions.

🧠 4. Présentation des principaux SGBD libres

4.1. MySQL

- **Origine :** Créé en 1995 par Michael Widenius (Suède).
- **Licence :** GPL, puis double licence (commerciale et libre) après son rachat par **Sun Microsystems** (2008), puis **Oracle Corporation** (2010).
- **Points forts :**
    - Rapidité d'exécution.
    - Simplicité d'installation et d'utilisation.
    - Très utilisé dans les applications web (LAMP : Linux, Apache, MySQL, PHP/Python).
- **Limites :**
    - Fonctionnalités avancées limitées (procédures, transactions) avant la version 5.
    - Ralentissement du développement open source depuis le rachat par Oracle.

4.2. PostgreSQL

- **Origine :** Dérivé du projet **Ingres**, lancé à l'Université de Californie (Berkeley) dans les années 1980.
- **Licence :** PostgreSQL License (type BSD, très permissive).
- **Points forts :**
    - Très conforme aux standards SQL.

- Supporte les **transactions complexes**, **triggers**, **fonctions** et **vues matérialisées**.
- Excellente extensibilité (types personnalisés, JSONB, PL/pgSQL).
- Très utilisé dans les applications d'entreprise et analytiques.

- **Limites :**
  - Moins simple à apprendre que MySQL.
  - Configuration initiale plus technique.

4.3. MariaDB

- **Origine :** Créée en 2009 par Michael Widenius, fondateur de MySQL, après le rachat de MySQL par Oracle.
- **Licence :** GPL (entièrement libre).
- **Objectif :** Offrir une **alternative 100 % open source** à MySQL.
- **Points forts :**
  - Compatible avec MySQL (même syntaxe SQL).
  - Améliorations notables : moteurs de stockage plus rapides, optimisation des requêtes, réplication avancée.
  - Maintenue par la **MariaDB Foundation**.
- **Limites :**
  - Moins populaire dans les environnements non techniques.
  - Quelques divergences de compatibilité avec MySQL à partir des versions récentes.

⚖️ 5. Comparaison synthétique

Critère	MySQL	PostgreSQL	MariaDB
Licence	GPL / Commerciale	PostgreSQL License (BSD)	GPL
Propriétaire actuel	Oracle Corporation	Communauté open source	MariaDB Foundation
Performance (lecture)	Excellente	Bonne	Excellente
Performance (écriture)	Moyenne	Excellente	Excellente
Conformité SQL	Moyenne	Très élevée	Bonne
Extensibilité	Limitée	Très forte	Moyenne
Public cible	Web, CMS	Entreprises, analystes	Web et data-intensive
Outils de gestion	PhpMyAdmin, Workbench	pgAdmin	HeidiSQL, DBeaver

🔍 6. Conclusion

Les **SGBD libres** ont profondément transformé la gestion des données dans les milieux académique et professionnel. Leur adoption massive repose sur :

- La **philosophie du libre**,
- La **qualité technique** des solutions,
- Et leur **capacité d'adaptation** à des contextes variés (web, cloud, analytique).

👉 Dans la suite du cours, nous passerons de la théorie à la **mise en pratique** :

- Installation et configuration des trois SGBD,
- Création et manipulation de bases de données,
- Administration, optimisation et sécurisation.

MODULE 2 – Installation et configuration de MySQL, PostgreSQL et MariaDB sous Windows

Introduction

L'installation et la configuration des systèmes de gestion de bases de données (SGBD) libres constituent la première étape essentielle vers leur utilisation efficace. Ce module vise à familiariser l'étudiant avec les environnements MySQL, PostgreSQL et MariaDB, trois SGBD relationnels open source largement utilisée dans l'industrie. Sous Windows, ces systèmes nécessitent une installation rigoureuse et une configuration correcte pour assurer la stabilité, la performance et la sécurité des bases de données. L'objectif est de permettre à chaque étudiant d'installer, configurer et vérifier le bon fonctionnement de ces trois serveurs sur son poste.

Objectifs pédagogiques

À la fin de ce module, l'étudiant sera capable de :

- Installer **MySQL**, **PostgreSQL** et **MariaDB** sous Windows.
- Comprendre la structure de répertoires et les composants installés.
- Démarrer, arrêter et gérer les services des SGBD.
- Accéder aux interfaces clients (CLI, Workbench, pgAdmin, HeidiSQL).
- Configurer les paramètres essentiels (ports, utilisateurs, encodage, sécurité de base).
- Vérifier le bon fonctionnement du serveur et la connexion des utilisateurs.

Contenu du module

1. Présentation générale des SGBD libres

- **MySQL** : développé initialement par MySQL AB, maintenant sous Oracle.
- **PostgreSQL** : système orienté objet, reconnu pour sa conformité SQL et son extensibilité.
- **MariaDB** : fork communautaire de MySQL, compatible mais plus ouvert.
- Points communs : moteur relationnel, client-serveur, langage SQL, connectivité réseau.

2. Téléchargement des distributions

- **MySQL** :
  🔗 https://dev.mysql.com/downloads/
  Choisir *MySQL Installer for Windows (.msi)*
- **PostgreSQL** :
  🔗 https://www.postgresql.org/download/windows/
  Choisir *EDB Installer* ou *PostgreSQL Portable*.
- **MariaDB** :
  🔗 https://mariadb.org/download/
  Sélectionner la version stable (par ex. 11.x) pour Windows 64 bits.

3. Installation pas à pas

a. MySQL

1. Lancer *MySQL Installer for Windows*
2. Choisir le mode *Developer Default* (inclut serveur, client, Workbench).
3. Sélectionner le chemin d'installation (ex : C:\Program Files\MySQL\MySQL Server 8.0)
4. Définir le mot de passe de l'administrateur **root**.
5. Lancer le service MySQL et tester la connexion avec **MySQL Workbench**.
6. Vérifier le port par défaut : **3306**.

b. PostgreSQL

1. Lancer *PostgreSQL Installer (EDB)*.
2. Sélectionner les composants : *Server, pgAdmin 4, Stack Builder*.
3. Définir le mot de passe pour l'utilisateur **postgres**.
4. Vérifier le port **5432** et le dossier de données (C:\Program Files\PostgreSQL\16\data).
5. Démarrer le service et se connecter via **pgAdmin 4**.

c. MariaDB

1. Exécuter le programme d'installation téléchargé.
2. Définir le mot de passe de **root** et laisser le port **3306** (ou différent de MySQL).

3. Démarrer le service MariaDB et tester la connexion via **HeidiSQL** ou **mysql CLI**.
4. Vérifier que le service démarre automatiquement avec Windows.

4. Configuration de base

- Modifier le fichier de configuration :
  - my.ini (MySQL et MariaDB)
  - postgresql.conf et pg_hba.conf (PostgreSQL)
- Paramètres essentiels :
  - Port de connexion
  - Répertoire de données
  - Encodage par défaut (utf8 ou UTF8)
  - Connexions distantes (bind-address, listen_addresses)
- Redémarrer le service après chaque modification.

5. Gestion des services

- **MySQL / MariaDB :**
  - Démarrer : net start mysql
  - Arrêter : net stop mysql
  - Console : mysql -u root -p
- **PostgreSQL :**
  - Démarrer via *Services.msc* ou pg_ctl start
  - Arrêter via pg_ctl stop
  - Console : psql -U postgres

6. Vérification et test de connexion

Créer une petite base de test :

```
CREATE DATABASE testdb;
USE testdb;
CREATE TABLE utilisateur (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(50),
    email VARCHAR(100)
);
```

Insérer une donnée et exécuter un SELECT * FROM utilisateur;

Vérifier la bonne exécution sur chaque SGBD.

Mini-TP : Installation complète des trois SGBD

Objectif :

Installer et configurer MySQL, PostgreSQL et MariaDB sur une machine Windows, puis créer une base simple sur chacun.

Étapes à suivre :

1. Télécharger et installer les trois systèmes.
2. Configurer un mot de passe administrateur pour chaque serveur.
3. Créer une base TEST_BD et une table ETUDIANT(id, nom, prenom, email) sur chacun.
4. Insérer deux enregistrements et exécuter une requête de sélection.
5. Capturer l'écran de chaque test (Workbench, pgAdmin, HeidiSQL).

Clé de sortie :

- Connaissance des répertoires d'installation.
- Bonne maîtrise des outils clients graphiques et en ligne de commande.
- Capacité à modifier les fichiers de configuration et à redémarrer les services.
- Première base fonctionnelle sur chaque SGBD.

Conclusion

Ce module introduit les fondations techniques nécessaires à toute implémentation de bases de données libres. Savoir installer, configurer et démarrer un serveur est la première compétence opérationnelle d'un administrateur de base de données. Ces compétences seront consolidées dans les modules suivants, où nous aborderons la création de schémas, la manipulation de données et l'optimisation des requêtes.

MODULE 3 – Rappel sur la modélisation conceptuelle et logique (MCD, MLD). Passage du modèle logique au schéma SQL

Introduction

Avant toute implémentation d'une base de données, il est indispensable de disposer d'un **modèle de conception clair et cohérent**. Ce modèle permet de représenter le système d'information de manière structurée, d'éviter les redondances et d'assurer la cohérence des données.

Le processus de conception d'une base de données passe généralement par trois étapes :

1. **Le Modèle Conceptuel de Données (MCD)** : représentation abstraite des entités, associations et cardinalités.
2. **Le Modèle Logique de Données (MLD)** : traduction du MCD en un modèle relationnel.
3. **Le Schéma SQL** : création physique des tables dans le SGBD.

Ce module permet donc de réviser la méthodologie de modélisation (Merise) et de comprendre comment passer d'un modèle conceptuel à un schéma SQL prêt à être exécuté.

Objectifs pédagogiques

À la fin de ce module, l'étudiant sera capable de :

- Comprendre et construire un **MCD** cohérent à partir d'un cahier des charges.
- Transformer un MCD en **MLD** (relations et clés).
- Déterminer les **clés primaires, étrangères et attributs** pertinents.
- Générer le **schéma relationnel SQL** à partir du modèle logique.
- Identifier les erreurs de conception les plus courantes.

Contenu du module

1. Rappel sur la méthode Merise

Merise est une méthode de conception des systèmes d'information qui distingue trois niveaux :

- **Conceptuel** : représentation des objets réels (MCD).
- **Logique** : traduction en relations adaptées au modèle relationnel (MLD).
- **Physique** : création dans le SGBD (schéma SQL).

Les principaux outils :

- **MCD** : entités, associations, attributs, cardinalités.
- **MLD** : relations, clés primaires et étrangères.

- **MPD** : tables, types de données, contraintes SQL.

2. Le Modèle Conceptuel de Données (MCD)

Le **MCD** représente les **entités** (objets d'intérêt), leurs **attributs** et les **associations** entre elles.

Éléments constitutifs :

- **Entité** : objet ayant une existence propre (ex : Étudiant, Cours, Professeur).
- **Attribut** : propriété décrivant l'entité (ex : nom, âge, matricule).
- **Identifiant** : attribut unique permettant d'identifier chaque instance.
- **Association** : lien entre plusieurs entités (ex : un étudiant suit un cours).
- **Cardinalité** : nombre minimum et maximum d'occurrences possibles dans la relation (ex : 1,n ou 0,n).

Exemple MCD simple :

Un **étudiant** s'inscrit à plusieurs **cours**, et chaque cours peut être suivi par plusieurs étudiants.
Chaque inscription indique la **date d'inscription**.

Représentation :

[ETUDIANT]——(s'inscrit)——[COURS]
(1,n) (0,n)

3. Passage au Modèle Logique de Données (MLD)

Le **MLD** transforme les entités et associations du MCD en **relations (tables)** du modèle relationnel.

Règles de transformation :

- Chaque **entité** devient une **table**.
- Chaque **attribut** devient une **colonne**.
- L'**identifiant** devient la **clé primaire (PRIMARY KEY)**.
- Une **association 1,n** : la clé primaire du côté « 1 » devient clé étrangère du côté « n ».
- Une **association n,n** : devient une **table d'association** contenant les clés primaires des deux entités.

Exemple :
À partir du MCD précédent :

Entité	Attributs
ETUDIANT	id_etu, nom, prenom
COURS	id_cours, libelle
S_INSCRIT	id_etu, id_cours, date_inscription

MLD obtenu :

ETUDIANT(id_etu, nom, prenom)

COURS(id_cours, libelle)

S_INSCRIT(id_etu, id_cours, date_inscription)

Avec :

- PK(ETUDIANT) = id_etu
- PK(COURS) = id_cours
- PK(S_INSCRIT) = (id_etu, id_cours)
- FK(S_INSCRIT.id_etu) ⟶ ETUDIANT.id_etu
- FK(S_INSCRIT.id_cours) ⟶ COURS.id_cours

4. Passage au Schéma SQL (implémentation)

Le **schéma SQL** correspond à la traduction du MLD en instructions SQL exécutables.

Exemple MySQL / PostgreSQL / MariaDB :

```
CREATE TABLE ETUDIANT (
   id_etu INT PRIMARY KEY,
   nom VARCHAR(50),
   prenom VARCHAR(50)
);

CREATE TABLE COURS (
   id_cours INT PRIMARY KEY,
   libelle VARCHAR(100)
```

```
);

CREATE TABLE S_INSCRIT (
   id_etu INT,
   id_cours INT,
   date_inscription DATE,
   PRIMARY KEY (id_etu, id_cours),
   FOREIGN KEY (id_etu) REFERENCES ETUDIANT(id_etu),
   FOREIGN KEY (id_cours) REFERENCES COURS(id_cours)
);
```

5. Bonnes pratiques de modélisation

- Nommer les tables et colonnes de manière claire et cohérente.
- Utiliser un même type pour les clés primaires et étrangères.
- Éviter les attributs répétitifs ou dépendants d'autres attributs.
- Normaliser le modèle (jusqu'à la 3e forme normale au moins).
- Toujours représenter les cardinalités avant de créer le MLD.

Mini-TP : De la modélisation au SQL

Énoncé :
Un lycée souhaite informatiser la gestion de ses cours.
Chaque **professeur** enseigne plusieurs **matières**.
Chaque **élève** suit plusieurs **matières**.
On veut enregistrer les **notes** obtenues par chaque élève dans chaque matière.

Travail à faire :

1. Construire le **MCD** avec les entités, attributs et associations.
2. Transformer le MCD en **MLD**.
3. Écrire le **schéma SQL** correspondant (CREATE TABLE + contraintes).
4. Vérifier la cohérence des clés et des cardinalités.

Clé de sortie :

- Capacité à modéliser un système réel à l'aide de Merise.
- Maîtrise de la traduction MCD → MLD → SQL.
- Première approche structurée d'un projet de base de données relationnelle.

Conclusion

La modélisation conceptuelle et logique constitue la fondation de tout projet de base de données bien structuré. Elle garantit la cohérence, la flexibilité et la durabilité du système d'information.
En comprenant le passage du MCD au MLD, puis au schéma SQL, l'étudiant devient capable de concevoir une base solide avant son implémentation technique dans MySQL, PostgreSQL ou MariaDB.

MODULE 4 – Création de bases et de tables, types de données, contraintes d'intégrité

Introduction

Après la modélisation du système d'information, l'étape suivante consiste à **implémenter physiquement la structure de la base** dans un SGBD. Cette opération passe par la **création des bases, des tables et de leurs relations**, en choisissant les types de données adéquats et en appliquant les contraintes d'intégrité nécessaires pour assurer la fiabilité des informations.

Dans MySQL, PostgreSQL et MariaDB, cette étape repose sur les commandes **DDL (Data Definition Language)** telles que CREATE, ALTER et DROP. Ce module vise à rendre l'étudiant autonome dans la création de schémas complets et cohérents à partir d'un modèle logique.

Objectifs pédagogiques

À la fin de ce module, l'étudiant sera capable de :

- Créer une base de données et des tables relationnelles dans les trois SGBD.
- Identifier et utiliser les différents **types de données SQL**.
- Définir les **clés primaires, étrangères et uniques**.
- Mettre en place les **contraintes d'intégrité** pour garantir la cohérence des données.
- Modifier ou supprimer des objets dans une base de données.

Contenu du module

1. Création d'une base de données

La création d'une base permet de définir un espace logique de stockage pour les tables, vues et procédures.

Syntaxe générale :

CREATE DATABASE nom_base;

Exemples :

- MySQL / MariaDB :
- CREATE DATABASE ECOLE CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
- PostgreSQL :
- CREATE DATABASE ECOLE WITH ENCODING 'UTF8' OWNER = postgres;

Pour utiliser la base :

```
USE ECOLE;  -- MySQL / MariaDB
\c ECOLE;   -- PostgreSQL
```

2. Création de tables

La commande CREATE TABLE permet de définir la structure d'une table : colonnes, types de données et contraintes.

Syntaxe générale :

```
CREATE TABLE nom_table (
    nom_colonne type_donnee [contraintes],
    ...
);
```

Exemple :

```
CREATE TABLE ETUDIANT (
    id_etu INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(50) NOT NULL,
    prenom VARCHAR(50),
    date_naissance DATE,
    email VARCHAR(100) UNIQUE
);
```

3. Types de données SQL

Chaque SGBD possède ses propres types, mais ils respectent une base commune.

Catégorie	MySQL / MariaDB	PostgreSQL	Description
Numérique	INT, DECIMAL, FLOAT	INTEGER, NUMERIC, REAL	Nombres entiers ou décimaux
Caractère	CHAR, VARCHAR	CHAR, VARCHAR, TEXT	Chaînes de caractères
Date / Heure	DATE, DATETIME, TIME	DATE, TIMESTAMP	Données temporelles
Booléen	TINYINT(1), BOOLEAN	BOOLEAN	Valeurs logiques
Binaire	BLOB	BYTEA	Données binaires
Autres	ENUM, SET	ENUM, JSONB	Types spécifiques

Exemple :

```
CREATE TABLE PRODUIT (
    id_prod SERIAL PRIMARY KEY,
    nom VARCHAR(100),
    prix DECIMAL(10,2),
    date_fab DATE,
    en_stock BOOLEAN DEFAULT TRUE
);
```

4. Contraintes d'intégrité

Les **contraintes** assurent la cohérence et la validité des données dans une base.

Type de contrainte	Mot-clé	Rôle
Clé primaire	PRIMARY KEY	Identifie chaque ligne de manière unique
Clé étrangère	FOREIGN KEY	Relie une table à une autre
Non nullité	NOT NULL	Empêche les valeurs vides
Unicité	UNIQUE	Interdit les doublons
Valeur par défaut	DEFAULT	Affecte une valeur initiale
Vérification	CHECK	Imposent des conditions logiques sur les données

Exemple :

```
CREATE TABLE INSCRIPTION (
    id_inscr INT PRIMARY KEY AUTO_INCREMENT,
    id_etu INT NOT NULL,
    id_cours INT NOT NULL,
    date_inscr DATE DEFAULT CURRENT_DATE,
    note DECIMAL(5,2) CHECK (note BETWEEN 0 AND 20),
    FOREIGN KEY (id_etu) REFERENCES ETUDIANT(id_etu),
    FOREIGN KEY (id_cours) REFERENCES COURS(id_cours)
);
```

5. Modification et suppression de tables

- **Ajouter une colonne :**
- ALTER TABLE ETUDIANT ADD COLUMN sexe CHAR(1);
- **Modifier un type ou une contrainte :**
- ALTER TABLE ETUDIANT MODIFY email VARCHAR(150);

(PostgreSQL : ALTER COLUMN email TYPE VARCHAR(150);)

- **Supprimer une table :**
- DROP TABLE INSCRIPTION;
- **Supprimer une base :**
- DROP DATABASE ECOLE;

6. Intégrité référentielle

Une **clé étrangère** garantit que les données liées dans deux tables sont cohérentes.

Exemple :
Si une inscription fait référence à un étudiant, cet étudiant doit exister dans la table ETUDIANT.
L'intégrité référentielle peut définir des actions automatiques :

```
FOREIGN KEY (id_etu)
  REFERENCES ETUDIANT(id_etu)
  ON DELETE CASCADE
  ON UPDATE CASCADE;
```

→ Supprime automatiquement les inscriptions liées si l'étudiant est supprimé.

Mini-TP : Création d'une base scolaire

Énoncé :
Créer une base de données nommée LYCEE_DB avec les tables suivantes :

- **ETUDIANT(id, nom, prenom, date_naissance, email)**
- **COURS(id, libelle, coefficient)**
- **INSCRIPTION(id, id_etu, id_cours, date_inscr, note)**

Travail à faire :

1. Créer la base et les tables avec les bons types et contraintes.
2. Ajouter les clés primaires et étrangères.
3. Insérer quelques données d'exemple.
4. Tester les contraintes (unicité, non-nullité, check sur la note).
5. Supprimer et modifier une colonne pour tester ALTER TABLE.

Clé de sortie :

- Maîtrise du DDL et de la création structurée de tables.
- Bonne utilisation des types de données et contraintes.
- Capacité à garantir la cohérence du schéma relationnel.

Conclusion

La création correcte des bases et des tables est le fondement de toute implémentation réussie.
Un schéma bien structuré, normalisé et contraint garantit la qualité et la fiabilité des données.
Ce module marque la transition entre la conception logique et la manipulation concrète de données, qui sera abordée dans le prochain module avec les commandes SQL de base.

MODULE 5 – Les requêtes SQL de base : SELECT, INSERT, UPDATE, DELETE

Introduction

Après avoir créé les bases et les tables, la prochaine étape est d'apprendre à **manipuler les données** qu'elles contiennent.

Le **langage SQL (Structured Query Language)** permet d'effectuer ces opérations grâce à un sous-ensemble appelé **DML (Data Manipulation Language)**.

Ce module aborde les quatre commandes fondamentales :

- INSERT → pour insérer des données,
- SELECT → pour interroger les données,
- UPDATE → pour les modifier,
- DELETE → pour les supprimer.

Ces opérations sont la base de tout travail sur les bases de données relationnelles et s'appliquent de manière similaire dans MySQL, PostgreSQL et MariaDB.

Objectifs pédagogiques

À la fin de ce module, l'étudiant sera capable de :

- Insérer de nouvelles lignes dans une table.
- Lire et filtrer les données avec différentes clauses (WHERE, ORDER BY, LIMIT).
- Mettre à jour une ou plusieurs lignes.
- Supprimer sélectivement des données.
- Comprendre la différence entre manipulation logique et physique des données.

Contenu du module

1. La commande INSERT – Insérer des données

La commande INSERT permet d'ajouter des enregistrements dans une table.

Syntaxe générale :

```
INSERT INTO nom_table (col1, col2, col3, ...)
VALUES (val1, val2, val3, ...);
```

Exemples :

```
INSERT INTO ETUDIANT (nom, prenom, date_naissance, email)
VALUES ('Kossi', 'Jean', '2002-05-17', 'jean.kossi@gmail.com');
```

```
INSERT INTO COURS (libelle, coefficient)
VALUES ('Mathématiques', 3), ('Informatique', 2);
```

💡 On peut insérer plusieurs lignes à la fois en séparant les blocs VALUES par des virgules.

2. La commande SELECT – Lire les données

SELECT permet d'interroger la base de données pour afficher tout ou partie des informations d'une table.

Syntaxe générale :

```
SELECT [DISTINCT] colonnes
FROM nom_table
[WHERE condition]
[ORDER BY colonne [ASC|DESC]]
[LIMIT n];
```

Exemples :

```
-- Afficher toutes les données
SELECT * FROM ETUDIANT;

-- Afficher le nom et l'email des étudiants
SELECT nom, email FROM ETUDIANT;

-- Afficher les étudiants nés après 2000
SELECT nom, prenom FROM ETUDIANT
WHERE date_naissance > '2000-01-01';

-- Trier les résultats par nom
SELECT * FROM ETUDIANT ORDER BY nom ASC;

-- Limiter à 3 résultats
SELECT * FROM ETUDIANT LIMIT 3;
```

Clauses utiles :

- WHERE → filtre les lignes selon une condition.

- DISTINCT → élimine les doublons.
- ORDER BY → trie les résultats.
- LIMIT (MySQL/MariaDB) ou FETCH FIRST n ROWS ONLY (PostgreSQL) → limite le nombre de résultats.

3. La commande UPDATE – Modifier des données

UPDATE permet de modifier une ou plusieurs lignes existantes dans une table.

Syntaxe générale :

```
UPDATE nom_table
SET col1 = val1, col2 = val2, ...
WHERE condition;
```

Exemples :

```
-- Modifier l'adresse email d'un étudiant
UPDATE ETUDIANT
SET email = 'jean.kossi@etu.tg'
WHERE id_etu = 1;

-- Augmenter la note de 2 points pour tous les étudiants
UPDATE INSCRIPTION
SET note = note + 2
WHERE note < 18;
```

⚠️ Sans clause WHERE, **toutes les lignes** de la table seront modifiées !

4. La commande DELETE – Supprimer des données

DELETE permet de retirer une ou plusieurs lignes d'une table.

Syntaxe générale :

```
DELETE FROM nom_table
WHERE condition;
```

Exemples :

```
-- Supprimer un étudiant
DELETE FROM ETUDIANT WHERE id_etu = 3;
```

```
-- Supprimer toutes les inscriptions avant 2023
DELETE FROM INSCRIPTION WHERE date_inscr < '2023-01-01';
```

Suppression complète d'une table (toutes les lignes) :

```
DELETE FROM ETUDIANT;
```

💡 Pour réinitialiser complètement une table et son compteur auto-incrémenté :

- MySQL/MariaDB : TRUNCATE TABLE ETUDIANT;
- PostgreSQL : TRUNCATE TABLE ETUDIANT RESTART IDENTITY;

5. Combinaisons pratiques

Les quatre commandes s'utilisent souvent ensemble dans un cycle de gestion :

1. INSERT pour créer de nouvelles données,
2. SELECT pour vérifier,
3. UPDATE pour corriger ou ajuster,
4. DELETE pour nettoyer ou archiver.

Exemple global :

```
INSERT INTO COURS (libelle, coefficient)
VALUES ('Physique', 2);

SELECT * FROM COURS;

UPDATE COURS
SET coefficient = 3
WHERE libelle = 'Physique';

DELETE FROM COURS
WHERE coefficient < 2;
```

Mini-TP : Manipulation de données

Énoncé :

Sur la base LYCEE_DB créée au module précédent :

1. Insérer 3 étudiants, 3 cours et 5 inscriptions.

2. Afficher tous les étudiants inscrits.
3. Afficher les cours dont le coefficient ≥ 2.
4. Modifier une note et mettre à jour l'adresse email d'un étudiant.
5. Supprimer un étudiant et toutes ses inscriptions.

Clé de sortie :

- Maîtrise des commandes DML.
- Capacité à combiner SELECT, INSERT, UPDATE et DELETE.
- Compréhension des effets des filtres WHERE et ORDER BY.
- Autonomie dans la manipulation des données d'une base relationnelle.

Conclusion

Les requêtes SQL de base constituent le cœur du langage de manipulation de données. Elles permettent de gérer efficacement le cycle de vie des informations dans une base. Une bonne maîtrise de ces commandes est indispensable avant d'aborder les requêtes plus complexes, qui feront l'objet du prochain module.

MODULE 6 – Requêtes complexes : jointures, agrégations, sous-requêtes

Introduction

Les bases de données relationnelles ne se limitent pas à de simples requêtes de sélection ou de mise à jour. Lorsqu'il s'agit de croiser les informations issues de plusieurs tables, de calculer des statistiques ou de filtrer des ensembles complexes de données, les **requêtes SQL avancées** deviennent indispensables. Ce module explore les **jointures**, les **agrégations** et les **sous-requêtes**, trois outils puissants pour interroger efficacement des bases MySQL, PostgreSQL et MariaDB.

Objectifs pédagogiques

À la fin de ce module, l'étudiant sera capable de :

- Comprendre le rôle et les types de jointures dans SQL.
- Effectuer des calculs statistiques à l'aide des fonctions d'agrégation et de la clause GROUP BY.
- Écrire et interpréter des sous-requêtes simples et corrélées.
- Manipuler les filtres avancés et les regroupements hiérarchiques.
- Appliquer ces concepts à des cas réels d'analyse de données.

Contenu du module

1. Les jointures (JOIN)

Les jointures permettent de combiner des lignes de plusieurs tables en fonction d'un lien logique.

- **INNER JOIN** : ne conserve que les enregistrements correspondants dans les deux tables.
- SELECT e.nom, d.nom_departement
- FROM employe e
- INNER JOIN departement d ON e.id_dept = d.id_dept;
- **LEFT JOIN (ou LEFT OUTER JOIN)** : conserve toutes les lignes de la table de gauche, même sans correspondance.
- **RIGHT JOIN (ou RIGHT OUTER JOIN)** : conserve toutes les lignes de la table de droite.
- **FULL JOIN (PostgreSQL uniquement)** : conserve toutes les lignes des deux tables, qu'il y ait correspondance ou non.

- **CROSS JOIN** : produit le produit cartésien des deux tables.
- **JOIN multiples** : possibilité de chaîner plusieurs jointures dans une même requête.

Clé de sortie : l'étudiant doit être capable d'écrire une requête de jointure à 2 ou 3 tables en comprenant la logique des clés primaires et étrangères.

2. Fonctions d'agrégation

Les agrégations permettent de calculer des valeurs statistiques à partir d'un ensemble de données.

Fonctions principales :

- COUNT() : compte les lignes.
- SUM() : somme les valeurs.
- AVG() : moyenne.
- MIN() et MAX() : valeurs extrêmes.

Exemple :

SELECT id_dept, COUNT(*) AS nb_employes, AVG(salaire) AS salaire_moyen

FROM employe

GROUP BY id_dept;

Clauses associées :

- GROUP BY : regroupe les données selon une colonne ou un ensemble de colonnes.
- HAVING : filtre les groupes après agrégation.
- SELECT id_dept, COUNT(*) AS nb
- FROM employe
- GROUP BY id_dept
- HAVING COUNT(*) > 5;

Clé de sortie : savoir analyser et manipuler des données regroupées en utilisant GROUP BY et HAVING.

3. Sous-requêtes (Subqueries)

Les sous-requêtes sont des requêtes imbriquées dans une autre requête principale.

- **Sous-requête dans WHERE :**
- SELECT nom
- FROM employe

- WHERE salaire > (SELECT AVG(salaire) FROM employe);
- **Sous-requête corrélée :**
- SELECT nom, salaire
- FROM employe e
- WHERE salaire > (SELECT AVG(salaire) FROM employe WHERE id_dept = e.id_dept);
- **Sous-requête dans FROM (table dérivée) :**
- SELECT dept, moyenne
- FROM (
- SELECT id_dept AS dept, AVG(salaire) AS moyenne
- FROM employe
- GROUP BY id_dept
-) AS t
- WHERE moyenne > 2000;

Clé de sortie : être capable d'écrire et de lire des sous-requêtes simples et corrélées, et comprendre leur impact sur les performances.

4. Combinaisons et filtres avancés

- Utilisation des opérateurs : IN, NOT IN, EXISTS, ANY, ALL.
- Comparaison entre plusieurs ensembles de résultats.
- Cas pratiques : recherche d'employés sans département, de clients sans commandes, etc.

Mini Travaux Pratiques (TP)

1. Créez deux tables :
   EMPLOYE(id_emp, nom, salaire, id_dept)
   DEPARTEMENT(id_dept, nom_dept)
2. Insérez quelques enregistrements.
3. Écrivez les requêtes suivantes :
   - Afficher le nom des employés et le nom de leur département.
   - Afficher le salaire moyen par département.
   - Lister les employés gagnant plus que la moyenne de leur département.

- Trouver les départements sans employés.
- Afficher les départements ayant plus de 3 employés.

Clés de sortie générales du module :

- Bonne maîtrise des jointures et des agrégations.
- Capacité à structurer une requête complexe et à interpréter ses résultats.
- Habileté à résoudre des problèmes d'analyse de données réelles par des sous-requêtes.

MODULE 7 – Vues, index et clés étrangères. Optimisation des requêtes

Introduction

Les systèmes de gestion de bases de données relationnelles (SGBD) offrent de nombreuses fonctionnalités pour **structurer, sécuriser et accélérer** l'accès aux données. Les **vues** permettent de simplifier les requêtes complexes ou de masquer certaines informations sensibles. Les **index** améliorent considérablement les performances de recherche et de tri. Les **clés étrangères** assurent la cohérence des données entre les tables liées.
Enfin, l'**optimisation des requêtes** vise à garantir un usage efficace des ressources du système.

Ce module montre comment utiliser ces outils pour concevoir des bases performantes, cohérentes et évolutives.

Objectifs pédagogiques

À la fin de ce module, l'étudiant sera capable de :

- Créer et gérer des **vues** pour simplifier les accès aux données.
- Comprendre le rôle et la création des **index**.
- Mettre en place des **clés étrangères** pour garantir l'intégrité référentielle.
- Identifier et corriger les problèmes de performance liés aux requêtes.
- Utiliser les outils d'**analyse et d'optimisation** disponibles dans MySQL, PostgreSQL et MariaDB.

Contenu du module

1. Les vues (VIEW)

Une **vue** est une table virtuelle issue d'une requête SQL enregistrée. Elle ne contient pas directement de données mais affiche celles des tables sous-jacentes.

Création d'une vue :

```
CREATE VIEW vue_employe_dept AS
SELECT e.nom, e.salaire, d.nom_dept
FROM employe e
JOIN departement d ON e.id_dept = d.id_dept;
```

Utilisation d'une vue :

```
SELECT * FROM vue_employe_dept WHERE salaire > 2000;
```

Mise à jour d'une vue :

CREATE OR REPLACE VIEW vue_employe_dept AS ...

Suppression :

DROP VIEW vue_employe_dept;

Avantages :

- Simplifie les requêtes complexes.
- Renforce la sécurité (restriction d'accès aux colonnes sensibles).
- Facilite la maintenance des applications.

Clé de sortie : savoir créer, utiliser et gérer des vues dans les trois SGBD.

2. Les index

Un **index** est une structure de données spéciale qui accélère la recherche, le tri et la jointure.

Création d'un index :

CREATE INDEX idx_nom ON employe(nom);

Index unique :

CREATE UNIQUE INDEX idx_unique_email ON employe(email);

Suppression d'un index :

DROP INDEX idx_nom;

(Sur PostgreSQL, la syntaxe complète est DROP INDEX IF EXISTS idx_nom;)

Quand utiliser un index ?

- Sur les colonnes souvent utilisées dans les clauses WHERE ou JOIN.
- Sur les colonnes servant au tri (ORDER BY).
- Éviter de créer trop d'index (ralentissent les insertions et mises à jour).

Clé de sortie : comprendre le compromis entre performance et coût d'entretien des index.

3. Les clés étrangères (FOREIGN KEY)

Les clés étrangères relient les tables entre elles et garantissent la cohérence des données.

Exemple :

```
CREATE TABLE departement (
  id_dept INT PRIMARY KEY,
  nom_dept VARCHAR(50)
```

```
);

CREATE TABLE employe (
  id_emp INT PRIMARY KEY,
  nom VARCHAR(50),
  id_dept INT,
  FOREIGN KEY (id_dept) REFERENCES departement(id_dept)
);
```

Options de contrainte :

- ON DELETE CASCADE : supprime les enregistrements liés.
- ON UPDATE CASCADE : met à jour automatiquement les valeurs modifiées.
- ON DELETE SET NULL : remplace la valeur par NULL si la référence disparaît.

Clé de sortie : savoir définir et manipuler les contraintes d'intégrité référentielle selon les besoins.

4. Optimisation des requêtes

L'optimisation vise à **améliorer la rapidité d'exécution** des requêtes SQL.

Bonnes pratiques :

- Utiliser des index appropriés.
- Éviter les sous-requêtes imbriquées inutiles (préférer les jointures).
- Limiter le nombre de colonnes sélectionnées (SELECT * à éviter).
- Supprimer les doublons avec discernement (DISTINCT ralentit les requêtes).
- Vérifier le plan d'exécution :
    - MySQL / MariaDB : EXPLAIN SELECT ...
    - PostgreSQL : EXPLAIN ANALYZE SELECT ...

Exemple :

```
EXPLAIN SELECT e.nom, d.nom_dept
FROM employe e
JOIN departement d ON e.id_dept = d.id_dept
WHERE d.nom_dept = 'Comptabilité';
```

Clé de sortie : interpréter le plan d'exécution et ajuster les requêtes pour améliorer les performances.

Mini Travaux Pratiques (TP)

1. Créez les tables **EMPLOYE** et **DEPARTEMENT** avec clé étrangère.
2. Créez un **index** sur le champ nom de la table **EMPLOYE**.
3. Créez une **vue** listant les employés et leur département.
4. Supprimez un employé et observez le comportement de la contrainte (CASCADE, SET NULL, etc.).
5. Comparez les temps d'exécution d'une requête **avec et sans index** à l'aide de EXPLAIN.

Clés de sortie du module

- Bonne compréhension de la **structure logique** et **physique** d'une base de données relationnelle.
- Capacité à **créer des vues sécurisées** et **optimiser les performances** par l'usage d'index.
- Maitrise de la **gestion des relations** entre tables par les clés étrangères.

MODULE 8 – Fonctions, procédures stockées et déclencheurs (Triggers)

Introduction

Les SGBD modernes offrent des mécanismes puissants pour automatiser le traitement des données côté serveur. Les **fonctions** et **procédures stockées** permettent de regrouper du code SQL réutilisable pour accomplir des tâches répétitives. Les **déclencheurs (triggers)** permettent, quant à eux, d'exécuter automatiquement des actions lors d'événements comme une insertion, une mise à jour ou une suppression.

Ce module introduit ces trois éléments essentiels à la programmation avancée des bases de données MySQL, PostgreSQL et MariaDB.

Objectifs pédagogiques

À la fin de ce module, l'étudiant sera capable de :

- Créer et exécuter des **fonctions** SQL pour encapsuler une logique de calcul.
- Définir et appeler des **procédures stockées** pour automatiser des opérations complexes.
- Créer des **déclencheurs (triggers)** pour garantir la cohérence et la traçabilité des données.
- Comprendre les avantages et limites de la logique côté serveur.

Contenu du module

1. Les fonctions

Une fonction retourne une valeur à partir d'un traitement défini par l'utilisateur.

Exemple :

```
CREATE FUNCTION Calcul_TVA(prix DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
  RETURN prix * 0.18;
END;
```

Utilisation :

```
SELECT nom, Calcul_TVA(prix) AS tva FROM produit;
```

2. Les procédures stockées

Contrairement à une fonction, une procédure peut effectuer plusieurs opérations sans nécessairement renvoyer une valeur.

Exemple :

```
DELIMITER //
CREATE PROCEDURE Augmenter_Salaire(IN pourcentage DECIMAL(5,2))
BEGIN
  UPDATE employe SET salaire = salaire + (salaire * pourcentage / 100);
END //
DELIMITER ;
```

Appel :

```
CALL Augmenter_Salaire(10);
```

3. Les déclencheurs (Triggers)

Un **trigger** s'exécute automatiquement avant ou après un événement (INSERT, UPDATE, DELETE).

Exemple :

```
CREATE TRIGGER log_suppression
AFTER DELETE ON employe
FOR EACH ROW
INSERT INTO journal_suppression(id_emp, date_suppression)
VALUES (OLD.id_emp, NOW());
```

Clé de sortie : comprendre et appliquer les mécanismes automatiques de gestion des données côté serveur.

Mini TP

1. Créez une fonction pour calculer le montant TTC d'un produit.
2. Créez une procédure pour augmenter les salaires d'un département.
3. Créez un trigger qui insère un enregistrement dans une table d'historique à chaque suppression d'un employé.

Clés de sortie

- Capacité à automatiser les traitements en SQL.
- Compréhension du rôle des fonctions, procédures et triggers.
- Capacité à assurer la traçabilité et la cohérence des opérations.

MODULE 9 – Transactions, verrouillage et gestion de la concurrence

Introduction

Les transactions garantissent la fiabilité des opérations sur une base de données, en s'assurant que toutes les actions liées à un traitement soient **exécutées complètement ou annulées** en cas d'erreur. Les mécanismes de **verrouillage** et de **gestion de la concurrence** permettent d'éviter les conflits entre utilisateurs simultanés.

Objectifs pédagogiques

L'étudiant apprendra à :

- Comprendre la notion de **transaction** et ses propriétés (ACID).
- Utiliser les commandes COMMIT, ROLLBACK, SAVEPOINT.
- Gérer les **verrous (locks)** et éviter les conflits d'accès.
- Comprendre la **gestion de la concurrence** et les niveaux d'isolation.

Contenu du module

1. Les transactions

Une transaction est un ensemble d'opérations considérées comme une unité de travail.

Syntaxe :

START TRANSACTION;

UPDATE compte SET solde = solde - 100 WHERE id = 1;

UPDATE compte SET solde = solde + 100 WHERE id = 2;

COMMIT;

Si une erreur survient :

ROLLBACK;

2. Les propriétés ACID

- **Atomicité** : tout ou rien.
- **Cohérence** : l'état final est valide.
- **Isolation** : les transactions simultanées ne se perturbent pas.
- **Durabilité** : les changements validés sont permanents.

3. Les verrous (Locks)

Les SGBD utilisent des verrous pour éviter les conflits d'accès aux lignes ou tables.

- **Verrou partagé (SHARED)** : plusieurs lectures simultanées.
- **Verrou exclusif (EXCLUSIVE)** : empêche toute autre opération concurrente.

Exemple :

SELECT * FROM employe FOR UPDATE;

4. Les niveaux d'isolation

- **READ UNCOMMITTED**
- **READ COMMITTED**
- **REPEATABLE READ**
- **SERIALIZABLE**

Plus le niveau est élevé, plus la cohérence est forte, mais au détriment des performances.

Mini TP

1. Créez deux sessions concurrentes et observez les effets d'une transaction non validée.
2. Utilisez SAVEPOINT pour créer des points de restauration intermédiaires.
3. Testez différents niveaux d'isolation dans PostgreSQL.

Clés de sortie

- Capacité à garantir la cohérence et la sécurité des transactions.
- Bonne compréhension du verrouillage et des niveaux d'isolation.
- Aptitude à gérer la concurrence dans des contextes multi-utilisateurs.

MODULE 10 – Gestion des utilisateurs et des privilèges. Sécurité des bases de données

Introduction

La sécurité des bases de données repose sur une bonne gestion des **utilisateurs**, des **droits d'accès** et des **autorisations**.

Les SGBD libres (MySQL, PostgreSQL, MariaDB) offrent des mécanismes fins de contrôle d'accès pour garantir la confidentialité, l'intégrité et la disponibilité des données.

Objectifs pédagogiques

À la fin de ce module, l'étudiant sera capable de :

- Créer et gérer des **comptes utilisateurs**.
- Accorder et révoquer des **privilèges** d'accès aux bases et objets.
- Comprendre la notion de **rôles** et de **groupes** d'utilisateurs.
- Mettre en œuvre des mesures de **sécurité** et de **chiffrement** au niveau des SGBD.

Contenu du module

1. Création et gestion des utilisateurs

MySQL / MariaDB :

```
CREATE USER 'etudiant'@'localhost' IDENTIFIED BY 'pass123';
```

PostgreSQL :

```
CREATE ROLE etudiant WITH LOGIN PASSWORD 'pass123';
```

Suppression :

```
DROP USER etudiant;
```

2. Attribution des privilèges

MySQL / MariaDB :

```
GRANT SELECT, INSERT ON base1.* TO 'etudiant'@'localhost';
REVOKE INSERT ON base1.* FROM 'etudiant'@'localhost';
```

PostgreSQL :

```
GRANT SELECT, UPDATE ON TABLE employe TO etudiant;
REVOKE UPDATE ON TABLE employe FROM etudiant;
```

3. Les rôles et groupes

Les rôles permettent de regrouper plusieurs privilèges ou utilisateurs.

Exemple :

```
CREATE ROLE gestionnaire;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO gestionnaire;
GRANT gestionnaire TO etudiant;
```

4. Mesures de sécurité supplémentaires

- Chiffrement des connexions (SSL/TLS).
- Sécurisation des mots de passe (PASSWORD_POLICY).
- Sauvegarde régulière et journalisation des accès.
- Protection contre les injections SQL (via les applications).

Mini TP

1. Créez trois utilisateurs : admin, gestionnaire, lecteur.
2. Donnez à chacun des privilèges adaptés à son rôle.
3. Testez les autorisations en exécutant des requêtes avec différents comptes.
4. Activez la connexion sécurisée SSL (dans un environnement de test).

Clés de sortie

- Maîtrise de la gestion des utilisateurs et des droits.
- Capacité à sécuriser une base de données dans un environnement multi-utilisateurs.
- Compréhension de la hiérarchie des rôles et des politiques de sécurité.

MODULE 11 – Sauvegarde et restauration des bases (mysqldump, pg_dump, etc.)

Introduction

La **sauvegarde et la restauration** des bases de données sont des opérations critiques pour garantir la **continuité des services**, la **protection contre la perte de données** et la **reprise après incident**. MySQL, MariaDB et PostgreSQL proposent des outils natifs pour effectuer ces tâches, comme mysqldump, pg_dump ou pg_restore. Ce module couvre les bonnes pratiques, les types de sauvegardes et les commandes essentielles.

Objectifs pédagogiques

À la fin de ce module, l'étudiant sera capable de :

- Identifier les différents types de sauvegardes (complète, incrémentale, logique, physique).
- Utiliser les outils natifs de MySQL, MariaDB et PostgreSQL pour sauvegarder et restaurer des bases.
- Planifier des sauvegardes régulières et automatisées.
- Tester l'intégrité des sauvegardes et assurer la récupération en cas de panne.

Contenu du module

1. Types de sauvegardes

- **Sauvegarde complète** : copie intégrale de la base.
- **Sauvegarde incrémentale** : seules les modifications depuis la dernière sauvegarde sont enregistrées.
- **Sauvegarde logique** : export du schéma et des données au format SQL (mysqldump, pg_dump).
- **Sauvegarde physique** : copie des fichiers du serveur de base de données (rapide mais dépend du SGBD).

2. Sauvegarde avec MySQL / MariaDB

Commande mysqldump :

```
mysqldump -u root -p nom_base > sauvegarde.sql
```

Sauvegarder toutes les bases :

```
mysqldump -u root -p --all-databases > sauvegarde_complete.sql
```

Options utiles :

- --routines : inclut les procédures et fonctions stockées.
- --triggers : inclut les triggers.
- --single-transaction : pour les tables InnoDB, évite les blocages.

Restauration :

mysql -u root -p nom_base < sauvegarde.sql

3. Sauvegarde avec PostgreSQL

Commande pg_dump :

pg_dump -U postgres -F c -b -v -f sauvegarde.backup nom_base

- -F c : format personnalisé.
- -b : inclut les données binaires.
- -v : mode verbeux.

Restauration avec pg_restore :

pg_restore -U postgres -d nom_base -v sauvegarde.backup

Sauvegarde complète de toutes les bases :

pg_dumpall -U postgres > sauvegarde_complete.sql

4. Bonnes pratiques

- Planifier des sauvegardes régulières (quotidiennes, hebdomadaires).
- Tester périodiquement la restauration.
- Conserver plusieurs générations de sauvegardes.
- Sécuriser les fichiers de sauvegarde (permissions, chiffrement).
- Documenter le processus et automatiser via scripts ou cron jobs.

Mini TP

1. Sauvegarder la base LYCEE_DB avec mysqldump.
2. Restaurer cette base dans un nouvel environnement de test.
3. Sauvegarder la même base avec pg_dump (PostgreSQL) et tester la restauration avec pg_restore.
4. Comparer les temps d'exécution et vérifier l'intégrité des données restaurées.

Clés de sortie

- Capacité à effectuer une sauvegarde complète et incrémentale.
- Maîtrise des commandes mysqldump, pg_dump et pg_restore.
- Compréhension de l'importance de la planification et de la sécurité des sauvegardes.
- Aptitude à restaurer une base en toute sécurité, même dans un environnement différent.

MODULE 12 – Administration : logs, tuning, monitoring

Introduction

L'administration d'une base de données implique le **suivi des performances**, la **surveillance des logs**, et le **tuning** pour optimiser l'utilisation des ressources serveur. MySQL, MariaDB et PostgreSQL offrent des outils et commandes pour contrôler l'état du système, détecter les erreurs, et ajuster les paramètres pour garantir des performances optimales.

Objectifs pédagogiques

À la fin de ce module, l'étudiant sera capable de :

- Lire et analyser les **logs** du SGBD.
- Comprendre et appliquer les techniques de **tuning**.
- Mettre en place des outils de **monitoring** pour détecter les anomalies et surveiller les performances.
- Appliquer les bonnes pratiques d'administration quotidienne.

Contenu du module

1. Les logs

- **Logs d'erreurs** : pour détecter les problèmes (fichiers error.log).
- **Logs généraux et slow query logs** : pour analyser les requêtes lentes.

Exemple MySQL/MariaDB :

SHOW VARIABLES LIKE 'slow_query_log';

SHOW VARIABLES LIKE 'long_query_time';

PostgreSQL :

- Fichier postgresql.log contient les erreurs et requêtes lentes.
- Paramètres : log_statement, log_min_duration_statement.

2. Tuning et optimisation

- Ajuster les **paramètres mémoire** (innodb_buffer_pool_size, work_mem).
- Analyser les **requêtes lentes** et optimiser les index.
- Ajuster le **nombre maximal de connexions** (max_connections).

Outils :

- MySQL/MariaDB : EXPLAIN, SHOW STATUS, mysqltuner.
- PostgreSQL : EXPLAIN ANALYZE, pg_stat_activity, pg_stat_statements.

3. Monitoring

- Surveiller la charge CPU, mémoire et disques.
- Alertes sur les erreurs critiques ou les requêtes bloquées.
- Outils complémentaires : Grafana, Prometheus, pgAdmin, Percona Monitoring Tools.

Mini TP

1. Activer et lire le **slow query log** sur MySQL.
2. Identifier les 5 requêtes les plus lentes et proposer des optimisations.
3. Utiliser pg_stat_activity pour identifier les transactions bloquées.

Clés de sortie

- Capacité à surveiller et diagnostiquer l'état de la base.
- Aptitude à améliorer les performances par tuning et optimisation.
- Compréhension des outils natifs de suivi et de reporting.

MODULE 13 – Comparaison fonctionnelle entre MySQL, PostgreSQL et MariaDB. Étude de cas

Introduction

Ce module compare les **fonctionnalités, performances et cas d'usage** des trois principaux SGBD libres : MySQL, MariaDB et PostgreSQL. Une étude de cas permet de choisir le SGBD le plus adapté à un projet donné.

Objectifs pédagogiques

- Connaître les différences et points communs des SGBD.
- Identifier les avantages et limitations de chaque système.
- Être capable de choisir un SGBD en fonction des besoins du projet.

Contenu du module

Fonctionnalité	MySQL	MariaDB	PostgreSQL
Moteur de stockage	InnoDB	InnoDB / Aria / XtraDB	natif MVCC
Transactions	Oui	Oui	Oui, MVCC complet
JSON support	Limité	Amélioré	Très complet (JSONB)
Vues matérialisées	Non natif	Non natif	Oui
Fonctions et triggers	Oui	Oui	Très avancé
Concurrence / Isolation	REPEATABLE READ	REPEATABLE READ	Serializable, Repeatable Read, etc.
Haute disponibilité	Réplication master-slave	Multi-source replication	Réplication logique et physique, streaming
Index et performances	Bon	Très bon	Excellent pour grandes données et requêtes complexes

Étude de cas pratique

- Projet e-commerce : haute écriture → MariaDB ou MySQL.
- Projet analytique ou gestion complexe → PostgreSQL.
- Besoin JSON et requêtes complexes → PostgreSQL.

Mini TP

1. Définir un projet fictif avec besoins transactionnels et analytiques.
2. Justifier le choix du SGBD le plus adapté.
3. Mettre en place un prototype simple dans le SGBD choisi.

Clés de sortie

- Capacité à comparer les SGBD sur critères fonctionnels.
- Aptitude à faire un choix justifié pour un projet réel.
- Compréhension des forces et limites des trois systèmes.

MODULE 14 – Cas pratique de synthèse : conception complète et implémentation

Introduction

Ce module met en pratique tous les acquis : modélisation, création de base, requêtes, sécurité, sauvegarde et administration.

Objectifs pédagogiques

- Concevoir une base complète selon un cahier des charges.
- Implémenter les tables, relations, vues, fonctions et triggers.
- Appliquer la sécurité et la gestion des utilisateurs.
- Tester les sauvegardes et la restauration.

Étude pratique

Exemple : Gestion d'une école

1. Tables : ETUDIANT, COURS, INSCRIPTION, PROFESSEUR.
2. Relations : clés primaires et étrangères.
3. Fonctions : calcul de la moyenne générale par étudiant.
4. Triggers : journalisation des inscriptions.
5. Sécurité : création de rôles Admin, Enseignant, Lecteur.
6. Sauvegarde et restauration de la base.

Clés de sortie

- Capacité à concevoir et implémenter un projet complet.
- Maîtrise pratique de toutes les commandes SQL et outils d'administration.
- Compréhension de l'intégration des différents modules du cours.

MODULE 15 – Évaluation finale et recommandations

Introduction

Ce dernier module vise à évaluer les compétences globales acquises et fournir des recommandations pour une mise en production sécurisée et performante.

Objectifs pédagogiques

- Tester la maîtrise des concepts théoriques et pratiques.
- Évaluer la capacité à résoudre des problèmes réels.
- Préparer à l'utilisation professionnelle des SGBD libres.

Évaluation

- **Théorique** : QCM et questions ouvertes sur SQL, transactions, sécurité.
- **Pratique** : projet complet de création, manipulation et optimisation d'une base.
- **Analyse critique** : choix du SGBD, justification des décisions techniques.

Clés de sortie

- Maîtrise complète des bases de données relationnelles libres.
- Capacité à concevoir, sécuriser et administrer une base en conditions réelles.
- Préparation à l'utilisation professionnelle dans des projets industriels ou académiques.